

软件工程课本核心内容

第1章 软件工程概述 1.1 软件的概念与分类 软件是计算机系统中与硬件相互依存的另一部分，是包括程序、数据及其相关文档的完整集合。其中，程序是按特定算法编写的计算机能识别和执行的指令序列；数据是程序运行所需的基础信息，用于描述客观事物的特征和状态；文档是与程序开发、维护和使用相关的图文资料，是保障软件可理解、可维护的重要支撑。与硬件不同，软件具有无形性、可复用性、无磨损性、易复制性、对环境依赖性强等特点，其生产过程不涉及物理加工，而是通过智力劳动实现逻辑构建。软件分类方式多样，按功能可分为系统软件、支撑软件和应用软件三大类。系统软件是管理和控制计算机硬件与软件资源的核心软件，为其他软件提供运行环境，常见的有操作系统（如 Windows、Linux、Unix）、编译程序、数据库管理系统等；支撑软件是协助软件开发、测试、维护和管理工具类软件，包括集成开发环境（IDE）、调试工具、版本控制工具（如 Git）、项目管理工具等；应用软件是为满足特定领域用户需求而开发的软件，覆盖生活、工作、生产等各个场景，例如办公软件（Word、Excel）、工业控制软件、金融管理软件、医疗信息化软件等。随着信息技术的快速

发展，软件的规模和复杂度不断提升，单一开发者凭借个人经验已无法完成大型软件的开发任务，软件工程学科应运而生，其核心目标是用工程化的方法规范软件开发过程，提高软件质量，降低开发成本，缩短开发周期。

1.2 软件工程的定义与目标 软件工程是一门研究如何用系统化、规范化、可度量的方法进行软件的开发、运行和维护的学科，它将工程学的原理和方法应用于软件开发领域，旨在解决软件危机。软件危机是 20 世纪 60 年代末至 70 年代初出现的一系列软件问题的总称，主要表现为：软件开发成本失控、开发周期大幅延长、软件质量低下、维护困难、缺乏统一的开发标准和规范，许多大型软件项目最终失败或无法达到预期目标。例如，IBM 公司在开发 OS/360 操作系统时，投入了大量的人力和物力，却因缺乏系统的管理和规范，导致项目延期、成本超支，成为软件危机的典型案例。软件工程的核心目标可概括为四个方面：一是提高软件质量，确保软件满足用户的功能需求和非功能需求（如可靠性、安全性、易用性、性能等）；二是降低开发成本，通过规范化流程和高效工具，减少人力、物力和时间投入；三是缩短开发周期，确保软件按时交付，满足市场和用户的及时需求；四是提升

软件可维护性,使软件在交付后能够方便地进行修改、升级和扩展,延长软件的生命周期。为实现上述目标,软件工程强调规范化、系统化和可度量性,要求在软件开发的各个阶段都遵循明确的流程、标准和方法,同时注重团队协作和工具支撑,将软件开发从“手工艺式”的个人行为转变为“工程化”的团队行为。

1.3 软件工程的基本原理与原则 经过多年的发展,软件工程形成了一系列公认的基本原理,这些原理为软件开发实践提供了指导,确保工程化方法的有效实施。

主要包括:

1. 用分阶段的生命周期计划严格管理: 将软件开发过程划分为多个明确的阶段,每个阶段都有明确的目标、任务、输出物和评审标准,通过制定详细的生命周期计划,对每个阶段的进度、成本和质量进行严格控制,避免项目失控。
2. 坚持进行阶段评审: 在软件开发的每个阶段结束后,进行正式的评审,检查该阶段的工作成果是否符合要求,及时发现和解决问题,避免问题积累到后续阶段,降低修改成本。
3. 实行严格的产品控制: 在软件开发过程中,需求、设计等文档和代码会不断发生变更,需要建立严格的变更控制流程,对变更进行申请、评估、审批和跟踪,确保变更的合理性和可控性,避免变更导致的混乱。
4. 采用现代程序设计技术: 积极采用先进

的程序设计方法和技术（如结构化设计、面向对象设计等），提高代码的可读性、可复用性和可维护性，减少开发过程中的错误。

5. 结果应能清楚地审查：软件开发的每个阶段的输出物（如需求规格说明书、设计文档、代码、测试报告等）都应清晰、规范，便于审查和理解，确保不同角色（开发人员、测试人员、维护人员、用户）之间的有效沟通。

6. 开发小组的人员应少而精：软件开发是一项智力密集型工作，小组人员过多会增加沟通成本和管理难度，应选择具备相应技能、经验丰富的人员组成精干的开发小组，提高工作效率。

7. 承认不断改进软件工程实践的必要性：软件工程是一门不断发展的学科，新技术、新方法和新工具不断涌现，开发团队应持续学习和改进，不断优化软件工程实践，提升开发能力。

基于上述原理，软件工程还遵循一些核心原则，包括模块化、抽象、信息隐藏、复用性、一致性等。模块化原则要求将软件系统分解为多个功能独立、接口清晰的模块，每个模块完成特定的功能，降低系统复杂度；抽象原则要求忽略次要细节，聚焦核心功能和本质特征，简化问题描述和解决方案；信息隐藏原则要求将模块的内部实现细节隐藏起来，只对外提供统一的接口，减少模块间的依赖，提高系统的可维护性；复用性原则

要求尽可能复用已有的软件组件、设计模式和方法，减少重复开发，提高开发效率和质量；一致性原则要求在软件开发的各个阶段，遵循统一的标准、规范和风格，确保文档和代码的一致性，便于沟通和维护。

1.4 软件工程的生命周期 软件生命周期是指软件从提出开发需求开始，经过开发、运行、维护，直到被淘汰的整个过程，它是软件工程管理 and 开发的基础框架。不同的软件工程方法对生命周期的阶段划分略有差异，但通常可分为以下六个核心阶段：

1. 可行性研究与计划阶段：这是软件生命周期的第一个阶段，主要任务是明确软件的开发目标和范围，对项目的技术可行性、经济可行性、操作可行性和法律可行性进行全面分析，制定项目计划，包括进度计划、成本计划、资源计划和质量计划等。该阶段的输出物主要有可行性研究报告和项目计划书。
2. 需求分析阶段：需求分析是确定用户对软件的具体需求的过程，核心任务是理解用户的意图，明确软件需要完成的功能和达到的性能指标，以及用户对软件的其他要求（如易用性、安全性、可扩展性等）。需求分析阶段需要与用户进行充分沟通，采用多种方法（如访谈、问卷调查、原型法等）收集需求，对需求进行分析、整理和验证，形成需求规格说明书。需求规格说明书是后续

开发工作的依据，必须清晰、完整、无歧义。

3. 软件设计阶段：软件设计是将需求规格说明书转化为具体的软件实现方案的过程，分为概要设计和详细设计两个子阶段。概要设计的核心任务是确定软件的整体架构，划分模块，定义模块间的接口和交互关系，确定数据结构和数据库设计方案；详细设计的核心任务是对每个模块的内部逻辑进行设计，确定模块的实现细节，包括算法、数据结构、接口实现等，生成详细设计说明书。

4. 编码阶段：编码阶段是根据详细设计说明书，使用特定的编程语言编写程序代码的过程，核心任务是将设计方案转化为可执行的程序，确保代码的正确性、可读性、可复用性和可维护性。编码过程中需要遵循编码规范，进行代码注释，便于后续的测试和维护。

5. 测试阶段：测试阶段是检验软件是否满足需求规格说明书要求的过程，核心任务是通过各种测试方法（如单元测试、集成测试、系统测试、验收测试等）发现软件中的错误和缺陷，对错误进行定位和修复，确保软件质量。测试阶段需要制定测试计划和测试用例，记录测试结果，生成测试报告。

6. 运行与维护阶段：软件交付用户使用后，进入运行与维护阶段，核心任务是保障软件的正常运行，处理用户反馈的问题，对软件进行修改、升级和扩展，以适

应环境变化和用户需求的变化。软件维护分为改正性维护、适应性维护、完善性维护和预防性维护四类，维护阶段是软件生命周期中持续时间最长、成本最高的阶段，通常占软件总成本的 60%~80%。软件生命周期的各个阶段相互关联、相互制约，前一个阶段的工作成果是后一个阶段的工作依据，每个阶段的评审工作确保了软件质量的逐步提升，避免了问题的积累和扩散。

第2章 软件过程模型 2.1 软件过程模型的概念 软件过程是指软件开发、运行和维护所涉及的一系列活动、方法、工具和实践的集合，而软件过程模型则是对软件过程的抽象描述，它规定了软件生命周期中各个阶段的顺序、活动和任务，为软件开发提供了标准化的框架和指导。不同的软件项目具有不同的特点（如规模、复杂度、需求稳定性等），需要选择合适的软件过程模型，以确保项目的顺利实施。软件过程模型的核心作用是：明确软件开发的流程和步骤，规范开发活动，提高开发效率，降低开发风险，确保软件质量。一个优秀的软件过程模型应具备灵活性、可扩展性和可适应性，能够根据项目的实际情况进行调整和优化。常见的软件过程模型包括瀑布模型、快速原型模型、增量模型、螺旋模型、敏捷模型等，每种模型都有其

自身的特点、适用场景和优缺点，开发团队需要根据项目需求、团队规模、技术水平等因素进行选择。

2.2 瀑布模型 瀑布模型是最早出现的软件过程模型，也是最经典、最基础的模型之一，由 Winston Royce 于 1970 年提出。该模型将软件生命周期划分为可行性研究与计划、需求分析、概要设计、详细设计、编码、测试、运行与维护七个连续的阶段，每个阶段的工作完成后，才能进入下一个阶段，如同瀑布一样自上而下、逐级流动，因此称为瀑布模型。瀑布模型的核心特点是：阶段划分明确，每个阶段有明确的目标和输出物；强调文档的完整性和规范性，每个阶段都需要生成详细的文档（如需求规格说明书、设计文档、测试报告等）；严格的阶段评审，每个阶段结束后必须经过评审，合格后才能进入下一阶段；开发过程呈线性顺序，不允许跨阶段进行，也不允许轻易回溯。瀑布模型的优点是：流程清晰、规范，易于理解和管理；强调文档的重要性，便于后续的维护和复用；适合需求明确、稳定，且规模较小、复杂度较低的软件项目。例如，小型工具软件、简单的管理系统等，采用瀑布模型能够快速、高效地完成开发。瀑布模型的缺点也较为明显：灵活性差，一旦需求发生变更，需要回溯到前面的阶段进行修改，修改成本高、周期长；无法

应对需求不明确的项目，因为在需求分析阶段如果无法准确把握用户需求，后续的开发工作都会受到影响；开发过程中用户参与度低，直到测试阶段或交付后，用户才能看到完整的软件，可能导致软件与用户实际需求不符；线性流程导致风险发现较晚，一旦某个阶段出现严重问题，可能导致整个项目延期或失败。由于瀑布模型的局限性，目前它主要用于需求明确、稳定的小型项目，对于大型、复杂、需求多变的项目，通常不推荐单独使用。

2.3 快速原型模型 快速原型模型是为解决瀑布模型中需求不明确的问题而提出的，其核心思想是：在软件开发的早期，快速构建一个能够反映用户核心需求的原型系统，让用户对原型进行试用和评价，根据用户的反馈意见对原型进行修改和完善，直到用户满意后，再基于原型进行正式的软件开发。快速原型模型的开发流程主要包括：需求收集与分析、原型设计与构建、原型试用与评价、原型修改与完善、正式开发五个阶段。其中，原型设计与构建阶段强调“快速”，不需要追求原型的完美性，只需实现用户的核心功能，能够让用户直观地了解软件的功能和界面即可；原型试用与评价阶段是核心，通过用户的试用，收集用户的反馈，明确用户的真实需求；原型修改与完善阶段

根据用户反馈对原型进行迭代修改，直到用户满意；最后，基于完善后的原型，进行正式的需求分析、设计、编码和测试，完成软件的开发。快速原型模型的优点是：能够快速获取用户的真实需求，避免因需求不明确导致的开发风险；用户参与度高，能够及时提出反馈，确保软件符合用户需求；灵活性强，能够根据用户反馈快速调整需求和设计，降低修改成本；缩短了需求分析阶段的时间，加快了项目进度。快速原型模型的缺点是：原型的构建可能会消耗一定的人力和物力，如果原型设计不合理，可能会导致后续开发工作的返工；容易让用户误解原型就是最终产品，对软件的质量和性能产生过高期望；对于大型、复杂的软件项目，原型的构建和修改难度较大，难以全面覆盖所有需求；原型开发过程中可能会忽略软件的非功能需求（如性能、安全性、可维护性等），导致正式开发时需要进行大量的补充和修改。快速原型模型适合需求不明确、用户需求变化频繁、规模中等的软件项目，例如，新领域的软件开发、用户界面设计要求较高的软件等。

2.4 增量模型 增量模型是一种将软件开发过程分解为多个增量模块，逐步开发和交付的过程模型。该模型将软件系统划分为多个相互独立的增量模块，每个

增量模块实现一部分功能，开发团队先开发核心的增量模块，交付给用户使用，然后再逐步开发后续的增量模块，不断完善软件系统，直到完成所有功能的开发。增量模型的开发流程主要包括：需求分析与增量划分、核心增量开发与交付、后续增量开发与交付、系统集成与测试四个阶段。其中，需求分析与增量划分阶段需要将软件的功能划分为多个增量模块，明确每个增量模块的功能、优先级和交付时间，核心增量模块通常是用户最急需的功能；核心增量开发与交付阶段开发核心模块，完成后交付用户使用，获取用户反馈；后续增量开发与交付阶段按照优先级依次开发其他增量模块，每次交付后都进行用户反馈和修改；最后，将所有增量模块进行集成和测试，确保整个软件系统的完整性和稳定性。增量模型的优点是：能够快速交付核心功能，满足用户的紧急需求，提高用户满意度；用户可以在使用过程中提出反馈，便于后续增量模块的优化和改进；降低了项目的风险，即使某个增量模块开发出现问题，也不会影响整个项目的进展；便于团队协作，不同的团队可以同时开发不同的增量模块，提高开发效率。增量模型的缺点是：需要对软件功能进行合理的增量划分，如果划分不合理，可能导致增量模块之间出现依赖关系，影响开发进度；

增量模块的集成难度较大，需要确保各个增量模块之间的接口兼容和数据一致；需要进行多次集成测试，增加了测试的工作量；如果用户对增量模块的需求发生变更，可能会影响后续增量模块的开发。增量模型适合需求明确、功能可拆分、规模较大、用户需要逐步使用软件的项目，例如，大型企业管理系统、电子商务平台等。

2.5 螺旋模型 螺旋模型是由 Barry Boehm 于 1988 年提出的，它结合了瀑布模型的系统性和结构化，以及快速原型模型的灵活性和迭代性，同时引入了风险分析机制，是一种面向风险的软件过程模型。该模型将软件生命周期划分为多个螺旋周期，每个螺旋周期都包括四个阶段：制定计划、风险分析、实施开发、评审与规划下一个周期，每个周期都比上一个周期更深入、更完善，逐步构建出完整的软件系统。螺旋模型的核心特点是：强调风险分析，在每个螺旋周期的开始，都需要对当前阶段可能存在的风险（如技术风险、成本风险、进度风险、需求风险等）进行识别、评估和应对，降低项目风险；采用迭代式开发，每个螺旋周期都是一次小型的软件开发过程，通过迭代逐步完善软件功能和质量；结合了瀑布模型的阶段划分和快速原型模型的用户反馈，既保证了开发的规范性，又提

高了灵活性；能够根据项目的实际情况调整螺旋周期的数量和每个周期的任务，适应性强。螺旋模型的开发流程：第一个螺旋周期，制定项目计划，识别初步风险，构建原型，评审后规划下一个周期；第二个螺旋周期，基于上一个周期的原型和反馈，进行需求分析和概要设计，识别风险并应对，开发核心功能，评审后规划下一个周期；后续螺旋周期依次进行详细设计、编码、测试等工作，不断优化软件，直到完成所有功能的开发和测试，交付用户使用。螺旋模型的优点是：具有很强的风险控制能力，能够及时识别和应对项目风险，降低项目失败的概率；灵活性高，能够根据用户反馈和风险变化调整开发计划和方案；适合大型、复杂、需求多变的软件项目，能够逐步完善软件功能和质量；强调迭代和评审，确保软件质量的持续提升。螺旋模型的缺点是：复杂度高，对开发团队的技术水平和项目管理能力要求较高；每个螺旋周期都需要进行风险分析和评审，增加了项目的管理成本和时间成本；开发周期较长，对于小型项目来说，可能显得过于繁琐；风险分析的准确性依赖于开发团队的经验，若风险分析不到位，可能导致项目风险失控。螺旋模型适合大型、复杂、高风险的软件项目，例如，航空航天软件、金融核心系统、大型工业控制软件等。

2.6 敏捷模型 敏捷模型是 20 世纪 90 年代末兴起的一种软件过程模型，源于对传统软件过程模型（如瀑布模型）的反思，其核心思想是“响应变化胜于遵循计划”，强调迭代开发、快速交付、用户参与和团队协作，注重软件的实际价值而非文档的完整性。敏捷模型不是一个单一的模型，而是一系列敏捷方法的集合，常见的有 Scrum、XP（极限编程）、Kanban（看板）等，这些方法虽然具体实践不同，但都遵循敏捷宣言的核心原则。敏捷宣言的核心原则包括：个体和交互胜过过程和工具；可以工作的软件胜过全面的文档；客户合作胜过合同谈判；响应变化胜过遵循计划。敏捷模型的核心特点是：迭代式开发，将软件开发过程分解为多个短周期的迭代（通常为 1~4 周），每个迭代都完成一部分功能，交付给用户使用，获取反馈；用户全程参与，与开发团队密切协作，及时提出需求和反馈，确保软件符合用户需求；团队自组织，开发团队自主决定如何完成迭代任务，提高团队的积极性和创造力；轻量级文档，只编写必要的文档，避免文档冗余，专注于软件的开发和交付；持续集成和持续测试，每次迭代都进行集成测试，及时发现和修复错误，确保软件质量；适应变化，能够快速响应用户需求的变化，调整开发计划和方案。以 Scrum 方法为例，

其核心流程包括：产品待办列表（Product Backlog），由产品负责人整理用户需求，按照优先级排序；冲刺计划会议（Sprint Planning），团队成员共同确定当前冲刺（迭代）的目标和任务；冲刺（Sprint），团队在规定的时间内完成冲刺任务，每天召开短会（Daily Scrum），同步进度、解决问题；冲刺评审会议（Sprint Review），向用户展示当前冲刺的成果，收集反馈；冲刺回顾会议（Sprint Retrospective），团队总结本次冲刺的经验和不足，优化后续的开发流程。敏捷模型的优点是：灵活性高，能够快速响应用户需求的变化，适应市场变化；用户参与度高，确保软件符合用户的实际需求；迭代周期短，能够快速交付可用的软件，提高用户满意度；团队协作效率高，自组织团队能够充分发挥成员的积极性和创造力；轻量级文档，减少了文档编写的工作量，专注于软件开发。敏捷模型的缺点是：缺乏严格的流程和规范，对于大型、复杂的软件项目，可能导致开发过程混乱，难以管理；对开发团队的技术水平和协作能力要求较高，若团队成员能力不足，可能影响项目进度和质量；轻量级文档可能导致后续维护困难，缺乏足够的文档支撑；需求变化过于频繁可能导致项目范围失控，增加开发成本和周期。敏捷模型适合需求多变、规模中等、团队协作

能力强的软件项目，例如，互联网产品、移动应用、小型管理系统等，目前已成为互联网行业最常用的软件过程模型之一。

第3章 需求工程 3.1 需求工程的概念与意义 需求工程是软件工程的核心组成部分，是指在软件生命周期中，围绕需求的获取、分析、建模、验证、管理等一系列活动的集合，其核心目标是准确、完整、无歧义地获取用户需求，并将其转化为可执行的软件需求规格说明书，为后续的设计、编码、测试等工作提供依据。需求是用户对软件的期望和要求，包括功能需求、非功能需求和约束条件。功能需求是软件必须完成的具体功能，例如，用户登录、数据查询、信息修改等；非功能需求是软件在运行过程中需要满足的质量属性，例如，性能（响应时间、并发量）、可靠性（平均无故障时间）、安全性（数据加密、权限控制）、易用性（操作简单、界面友好）、可扩展性（支持功能扩展）等；约束条件是软件开发过程中需要遵循的限制，例如，编程语言、硬件环境、开发周期、预算等。需求工程的意义重大，高质量的需求是软件成功的关键。如果需求获取不准确、不完整，会导致后续的设计、编码、测试等工作出现偏差，最终开发出的软件不符合用户需求，造成人力、物力和时间的浪费，甚至导

致项目失败。反之，完善的需求工程能够明确软件的开发目标和范围，减少需求变更，降低开发风险，提高软件质量，确保项目顺利实施。需求工程的整个过程贯穿软件生命周期的多个阶段，从可行性研究阶段的需求初步调研，到需求分析阶段的详细需求收集和分析，再到后续的需求验证、需求变更管理，直到软件维护阶段的需求迭代，需求工程始终发挥着核心指导作用。

3.2 需求获取 需求获取是需求工程的第一步，也是最关键的一步，其核心任务是通过各种方法，从用户、客户、领域专家等利益相关者那里收集软件的需求信息，明确用户的真实期望和要求。需求获取的质量直接影响后续的需求分析和建模工作，因此，需要采用科学、有效的方法，确保获取的需求准确、完整、无歧义。常见的需求获取方法包括：

1. 访谈法：这是最常用的需求获取方法，分为结构化访谈和非结构化访谈。结构化访谈是提前准备好访谈提纲，按照提纲向利益相关者提问，收集需求信息，适合需求较为明确的场景；非结构化访谈没有固定的提纲，围绕软件的核心目标与利益相关者进行自由交流，适合需求不明确、需要深入了解用户需求的场景。访谈对象应覆盖不同类型的利益相关者，包括用户、客户、领域专

家、管理人员等，确保需求的全面性。

2. 问卷调查法：通过设计调查问卷，向大量的用户收集需求信息，适合用户数量较多、需求较为分散的场景。调查问卷的设计应简洁、明确，避免模糊不清的问题，问题类型可分为选择题、填空题、简答题等，便于数据的统计和分析。

3. 观察法：通过观察用户的实际工作流程和操作习惯，了解用户的真实需求和使用场景，适合用户无法清晰描述需求的场景。观察法需要在自然的工作环境中进行，记录用户的操作步骤、遇到的问题和潜在的需求，确保获取的需求符合实际使用情况。

4. 原型法：如快速原型模型中所述，通过快速构建原型系统，让用户试用，收集用户的反馈意见，明确用户的需求。原型法适合需求不明确、用户对界面和功能有较高要求的场景，能够帮助用户直观地表达自己的需求。

5. 工作坊法：组织利益相关者、开发团队成员召开工作坊会议，围绕软件的需求进行讨论和交流，共同明确需求。工作坊法能够促进不同角色之间的沟通，解决需求中的分歧，确保需求的一致性和全面性。

6. 文档分析法：通过分析现有的相关文档（如业务流程文档、用户手册、竞品分析报告等），获取需求信息，适合有相关文档可参考的场景。文档分析法能够快速了解业务背景和现有系统的情况，补充其

他获取方法的不足。在需求获取过程中，需要注意以下几点：一是要与利益相关者建立良好的沟通关系，确保信息传递的准确性；二是要避免主观臆断，尊重用户的真实需求，不随意添加自己的理解；三是要及时记录需求信息，避免遗漏；四是要关注需求的优先级，区分核心需求和次要需求，为后续的开发计划提供依据。

3.3 需求分析 需求分析是在需求获取的基础上，对收集到的需求信息进行整理、分析、提炼和验证，明确需求之间的关系，消除需求中的矛盾、歧义和冗余，形成清晰、完整、无歧义的需求规格说明书的过程。需求分析的核心任务是“理解需求”和“规范需求”，将用户的自然语言描述转化为计算机能够理解的、规范化的需求描述。需求分析的主要步骤包括：

1. 需求整理与分类：将收集到的需求信息进行整理，按照功能需求、非功能需求和约束条件进行分类，去除重复的需求，梳理需求之间的关联关系。
2. 需求分析与提炼：对整理后的需求进行深入分析，明确每个需求的具体内容、实现目标和约束条件，提炼出核心需求，消除需求中的模糊性和歧义性。例如，用户提出“系统要快速响应”，需要进一步分析“快速响应”的具体指标，如响应时间不超过 2 秒、并发量不低于

1000 人/分钟等，使需求更加具体、可度量。

3. 需求建模：

采用合适的建模方法，将需求转化为可视化的模型，便于开发团队和利益相关者理解和沟通。常见的需求建模方法包括结构化建模方法（如数据流图、实体关系图）和面向对象建模方法（如用例图、类图、状态图）。数据流图（DFD）是结构化建模的核心工具，用于描述系统中数据的流动过程，包括外部实体、数据存储、处理模块和数据流四个基本元素。外部实体是系统之外的人员或设备，如用户、客户、其他系统等；数据存储是用于存储数据的地方，如数据库、文件等；处理模块是对数据进行加工和处理的单元，如登录验证、数据查询等；数据流是数据的流动路径，描述数据从一个元素到另一个元素的传递过程。数据流图可以逐层分解，从顶层的系统整体数据流图，到中层的子系统数据流图，再到底层的详细数据流图，逐步细化系统的功能和数据流程。实体关系图（ERD）用于描述系统中数据之间的关系，包括实体、属性和关系三个基本元素。实体是系统中需要管理的对象，如用户、订单、商品等；属性是实体的特征，如用户的姓名、年龄、账号等；关系是实体之间的关联，如用户和订单之间的“购买”关系、商品和订单之间的“包含”关系等。ERD 是数据库设计的基础，能够帮

助开发团队明确数据结构和数据之间的关联。用例图是面向对象建模的核心工具，用于描述用户与系统之间的交互过程，包括参与者、用例和关系三个基本元素。参与者是与系统交互的人员或设备，如用户、管理员等；用例是系统为参与者提供的具体功能，如用户登录、订单查询、商品下单等；关系包括关联关系、包含关系、扩展关系等，描述参与者与用例、用例与用例之间的联系。用例图能够直观地展示系统的功能范围和用户的交互场景，帮助开发团队和用户理解系统的功能。

4. 需求验证：需求验证是确保需求规格说明书准确、完整、无歧义、可实现、可验证的过程，主要通过评审会议、原型演示、用户确认等方式进行。评审会议由开发团队、用户、领域专家等共同参与，对需求规格说明书进行审查，检查需求是否符合要求，是否存在矛盾、歧义或遗漏；原型演示通过展示原型系统，让用户确认需求是否符合自己的期望；用户确认是让用户在需求规格说明书上签字确认，明确需求的最终版本。需求分析阶段的输出物是需求规格说明书（SRS），它是后续开发工作的核心依据，应包含以下内容：引言（项目背景、开发目标、范围等）、总体描述（系统概述、运行环境、用户特征等）、功能需求（每个功能的详细描述、输入输出、处理流程

等)、非功能需求(性能、可靠性、安全性、易用性等)、约束条件、数据需求(数据结构、数据来源、数据存储等)、接口需求(与其他系统的接口描述)等。

3.4 需求建模方法 需求建模是需求分析的核心手段,通过建立可视化的模型,将抽象的需求转化为具体、可理解的表示形式,便于开发团队和利益相关者沟通和理解。需求建模方法主要分为结构化建模方法和面向对象建模方法两大类,两类方法各有特点,适用于不同的场景。

3.4.1 结构化建模方法 结构化建模方法源于结构化开发方法,其核心思想是将系统分解为多个功能模块,通过数据流和控制流描述系统的工作过程,注重系统的功能分解和流程描述。结构化建模方法主要包括数据流图(DFD)、实体关系图(ERD)、结构图等,其中,数据流图和实体关系图是最常用的工具。

数据流图(DFD)的构建步骤:首先确定系统的边界,明确系统的输入和输出;然后识别系统的外部实体、数据存储、处理模块和数据流;绘制顶层数据流图,描述系统与外部实体之间的交互;接着逐层分解处理模块,绘制中层和底层数据流图,细化每个处理模块的功能和数据流程;最后检查数据流图的正确性和完整性,确保没有遗漏的数据流和处理模块,

数据流的流向合理。 实体关系图 (ERD) 的构建步骤：首先识别系统中的实体，明确实体的名称和含义；然后确定每个实体的属性，区分主键和非主键属性；接着识别实体之间的关系，确定关系的类型（一对一、一对多、多对多）；最后绘制 ERD 图，标注实体、属性和关系，确保数据关系的准确性。 结构化建模方法的优点是：流程清晰、规范，便于理解和管理；适合需求明确、流程固定的系统，如业务流程相对稳定的管理系统、工业控制软件等；能够快速明确系统的数据流和数据结构，为后续的设计和编码提供依据。缺点是：缺乏对系统对象的封装和复用，难以应对需求变化；不适合复杂的、面向对象的系统；难以描述系统的动态行为和交互过程。

3.4.2 面向对象建模方法

面向对象建模方法源于面向对象开发方法，其核心思想是围绕系统中的对象进行建模，将对象的属性和行为封装在一起，注重系统的结构和交互，能够更好地应对需求变化。面向对象建模方法主要采用统一建模语言 (UML)，UML 是一种标准化的建模语言，包含多种图表，用于描述系统的不同方面，常见的有用于需求建模的用例图、类图、状态图、活动图等。用例图主要用于描述用户与系统之间的交互，明确系统的功能范围，如前所述，其核心元素包括参与者、用例

和关系。类图用于描述系统中的类、类的属性和方法，以及类之间的关系（继承、关联、聚合、组合等），是面向对象建模的核心图表。状态图用于描述对象在生命周期中的状态变化，以及触发状态变化的事件，适合描述具有复杂状态变化的对象。活动图用于描述系统的业务流程和操作步骤，展示活动之间的先后顺序和并行关系，便于理解系统的动态行为。面向对象建模方法的优点是：符合人类的思维习惯，便于理解和沟通；强调对象的封装、继承和多态，提高了软件的可复用性和可维护性；能够更好地应对需求变化，适合复杂的、面向对象的系统；能够全面描述系统的静态结构和动态行为。缺点是：建模过程相对复杂，对开发团队的技术水平要求较高；对于简单的、流程固定的系统，可能显得过于繁琐；建模过程中需要投入较多的时间和精力。在实际的需求建模过程中，开发团队可以根据项目的特点，结合结构化建模方法和面向对象建模方法的优点，选择合适的建模工具和方法，确保需求模型的准确性和完整性。

3.5 需求管理 需求管理是指在软件生命周期中，对需求的获取、分析、建模、验证、变更等过程进行全面的管理和控制，确保需求的一致性、完整性和可追溯性，避免需求变更导致的项目混乱，保障项目顺利实

施。需求管理贯穿软件生命周期的全过程，其核心任务包括需求变更管理、需求跟踪管理和需求评审管理。

3.5.1 需求变更管理 需求变更是软件开发过程中不可避免的，由于用户需求的变化、市场环境的变化、技术的发展等原因，可能会导致已确定的需求需要修改、补充或删除。如果需求变更缺乏有效的管理，会导致项目范围失控、开发成本增加、进度延期、软件质量下降等问题。因此，建立严格的需求变更管理流程是非常必要的。需求变更管理流程主要包括：

1. 变更申请：由利益相关者（用户、客户、开发团队等）提出需求变更申请，说明变更的原因、内容、优先级和影响范围，并提交给需求管理员。
2. 变更评估：需求管理员组织开发团队、测试团队、项目管理人员等对变更申请进行评估，分析变更的必要性、可行性、对项目进度、成本、质量的影响，以及变更的优先级。
3. 变更审批：根据评估结果，由项目负责人或相关决策人员对变更申请进行审批，决定是否批准变更。对于重大变更，可能需要召开评审会议，由多方共同决策。
4. 变更实施：如果变更申请获得批准，需求管理员将变更内容通知开发团队、测试团队等相关人员，开发团队根据变更内容修改需求规格说明书、设计文档、代码等，测试团队修改测试用例，实施变更。
- 5.

变更验证：变更实施完成后，测试团队对变更内容进行测试，确保变更符合要求，没有引入新的错误；需求管理员组织相关人员对变更结果进行验证，确认变更已正确实施。

6. 变更归档：将变更申请、评估报告、审批意见、变更实施记录等相关文档进行归档，便于后续的查询和追溯。

在需求变更管理过程中，需要注意以下几点：一是要建立变更控制委员会 (CCB)，负责变更的审批和决策；二是要对变更进行分类和优先级排序，优先处理核心需求的变更，避免无关的、低优先级的变更影响项目进度；三是要及时沟通，将变更的相关信息通知所有相关人员，确保各方对变更的理解一致；四是要记录变更的全过程，确保变更的可追溯性。

3.5.2 需求跟踪管理

需求跟踪管理是指建立需求与后续开发成果（如设计文档、代码、测试用例等）之间的关联关系，确保每个需求都能够被正确地实现和验证，同时也能够追溯到需求的来源和变更历史。

需求跟踪管理分为正向跟踪和反向跟踪：正向跟踪是从需求出发，跟踪需求在设计、编码、测试等阶段的实现情况，确保每个需求都被落实；反向跟踪是从设计、编码、测试等成果出发，追溯到对应的需求，确保所有开发成果都有需求依据，没有多余的功能。

需求跟踪管理通常通过需求跟踪矩阵来实现，需

需求跟踪矩阵是一种表格，记录了需求的标识、名称、来源、对应的设计文档、代码模块、测试用例等信息，便于跟踪和管理。需求跟踪矩阵的建立和维护能够帮助开发团队及时发现需求遗漏或实现偏差，确保软件符合需求规格说明书的要求。

3.5.3 需求评审管理

需求评审管理是指在需求工程的各个阶段，对需求相关的文档（如需求规格说明书、需求模型等）进行评审，确保文档的准确性、完整性、无歧义性和可实现性。需求评审分为阶段性评审和最终评审：阶段性评审是在需求获取、分析、建模等每个阶段结束后进行，检查该阶段的工作成果是否符合要求，及时发现和解决问题；最终评审是在需求规格说明书完成后进行，由开发团队、用户、领域专家等共同参与，对需求规格说明书进行全面审查，确认需求的最终版本，为后续的开发工作提供依据。需求评审的主要内容包括：需求是否完整，是否有遗漏；需求是否明确，是否存在歧义；需求是否可行，是否能够在规定的时间、成本和技术条件下实现；需求是否一致，是否存在矛盾；需求是否符合用户的实际需求；需求文档是否规范、清晰，便于理解和使用。需求评审的流程包括：制定评审计划，明确评审的目标、范围、参与人员和时间；分发评审文档，让参与评审的人员提前熟悉文档内容；

召开评审会议，由需求负责人介绍需求文档，参与评审的人员提出意见和建议；记录评审意见，对意见进行整理和分析，确定修改方案；修改需求文档，完成后进行复查，确保评审意见得到落实；归档评审记录和修改后的需求文档。

第4章 软件设计 4.1 软件设计的概念与目标 软件设计是软件工程的核心阶段之一，是将需求规格说明书转化为具体的软件实现方案的过程，其核心任务是确定软件的结构、模块、接口和数据结构，为后续的编码和测试工作提供详细的指导。软件设计的质量直接影响软件的性能、可维护性、可复用性和可靠性，是决定软件质量的关键环节。软件设计的目标可以概括为以下几点：

1. 满足需求：软件设计必须严格遵循需求规格说明书的要求，确保设计方案能够实现所有的功能需求和非功能需求，满足用户的期望。
2. 降低复杂度：通过合理的模块划分和结构设计，将复杂的软件系统分解为多个功能独立、接口清晰的模块，降低系统的复杂度，便于开发、测试和维护。
3. 提高可维护性：设计方案应具有良好的可读性、可扩展性和可修改性，便于后续的维护工作，减少维护成本。
4. 提高可复用性：通过模块化设计、抽象设计和设计模式的应用，提高软件组件的可复用性，减少重复开

发，提高开发效率。 5. 确保性能：设计方案应考虑软件的性能需求，如响应时间、并发量、资源占用等，通过合理的算法设计、数据结构设计和架构设计，确保软件的性能达到要求。 6. 保证安全性：设计方案应考虑软件的安全性需求，通过权限控制、数据加密、输入验证等设计，防止安全漏洞，保护用户数据和系统安全。软件设计分为概要设计和详细设计两个连续的子阶段，概要设计关注软件的整体架构和模块划分，详细设计关注每个模块的内部实现细节，两个阶段相互关联、相互制约，共同构成完整的软件设计过程。

4.2 概要设计 概要设计又称总体设计，是软件设计的第一个子阶段，其核心任务是确定软件的整体架构，划分模块，定义模块间的接口和交互关系，确定数据结构和数据库设计方案，制定设计规范和标准，为详细设计提供依据。概要设计的输出物是概要设计说明书，主要包括软件架构设计、模块划分、接口设计、数据结构设计、数据库设计等内容。

4.2.1 软件架构设计 软件架构是软件系统的整体结构，包括系统的模块划分、模块间的关系、数据流向和控制流向等，是软件设计的核心。软件架构设计的核心任务是选择合适的架构风格，确定系统的整体结构，确保系统的性能、可维护性、可扩展性等非功能需求得到满足。常

见的软件架构风格包括：

1. 分层架构：也称为层次架构，是最常用的软件架构风格之一，将软件系统分为多个层次，每个层次负责特定的功能，层次之间通过接口进行通信，上层依赖下层，下层为上层提供服务。常见的分层架构分为四层：表现层（负责用户交互，如界面展示、输入输出）、业务逻辑层（负责核心业务逻辑的处理）、数据访问层（负责与数据库的交互，如数据的查询、插入、修改、删除）、数据存储层（负责数据的存储，如数据库、文件等）。分层架构的优点是结构清晰、易于理解和维护；模块间的耦合度低，便于模块的替换和扩展；适合大型、复杂的软件系统。缺点是层次之间的通信开销较大，可能影响系统性能；对于简单的系统，可能显得过于繁琐。
2. 客户端-服务器架构（C/S 架构）：将软件系统分为客户端和服务端两部分，客户端负责用户交互和部分业务逻辑处理，服务器负责数据存储和核心业务逻辑处理，客户端通过网络与服务端进行通信。C/S 架构的优点是客户端功能强大，能够提供良好的用户体验；服务器集中管理数据，安全性高；适合对交互性和性能要求较高的系统，如桌面应用、工业控制软件等。缺点是客户端需要安装和维护，维护成本高；系统的扩展性较差，增加新的客户端需要进行额外的配置和

开发；对网络环境的要求较高。

3. 浏览器-服务器架构（B/S 架构）：是在 C/S 架构的基础上发展起来的，将软件系统分为浏览器、服务器两部分，客户端不需要安装专门的软件，通过浏览器访问服务器，服务器负责所有的业务逻辑处理和数据存储。B/S 架构的优点是客户端无需安装和维护，使用方便；系统的扩展性好，增加新的功能只需修改服务器端代码；适合互联网应用、网页应用等，如电子商务平台、在线办公系统等。缺点是对浏览器的兼容性要求较高；用户体验不如 C/S 架构；服务器的压力较大，对服务器的性能要求较高。

4. 微服务架构：将软件系统分解为多个独立的微服务，每个微服务负责一个特定的业务功能，微服务之间通过网络进行通信（如 RESTful API），每个微服务可以独立部署、升级和扩展，采用分布式架构进行管理。微服务架构的优点是灵活性高，能够快速响应需求变化；可扩展性好，能够根据业务需求灵活增加或减少微服务；容错性强，单个微服务出现故障不会影响整个系统的运行；适合大型、复杂、需求多变的互联网应用，如大型电商平台、社交平台等。缺点是架构复杂，对项目管理和技术水平要求较高；微服务之间的通信开销较大，需要解决分布式事务、服务注册与发现等问题；维护成本高，需要管理多个

微服务的部署和运行。选择软件架构风格时，需要考虑项目的需求、规模、复杂度、技术水平、维护成本等因素，确保架构风格能够满足软件的功能需求和非功能需求。

4.2.2 模块划分 模块是软件系统中功能独立、接口清晰的最小单元，模块划分是概要设计的核心任务之一，其核心目标是将软件系统分解为多个模块，使每个模块的功能单一、接口清晰，降低模块间的耦合度，提高模块的内聚度。内聚度是指模块内部各个元素之间的关联程度，内聚度越高，模块的独立性越强，可维护性和可复用性越好。常见的内聚类型从高到低依次为：功能内聚（模块内的所有元素都为实现同一个功能服务，是最高级别的内聚）、顺序内聚（模块内的元素按照顺序执行，前一个元素的输出是后一个元素的输入）、通信内聚（模块内的元素都围绕同一个数据进行操作）、过程内聚（模块内的元素按照特定的过程执行）、时间内聚（模块内的元素在同一时间执行）、逻辑内聚（模块内的元素属于同一逻辑类别，但功能不相关）、偶然内聚（模块内的元素之间没有任何关联，是最低级别的内聚）。在模块划分时，应尽量追求功能内聚，避免偶然内聚和逻辑内聚。耦合度是指模块之间的关联程度，耦合度越低，模块的独

立性越强，模块间的相互影响越小，便于开发、测试和维护。常见的耦合类型从低到高依次为：无耦合（模块之间没有任何关联）、数据耦合（模块之间通过数据传递进行通信，是最低级别的耦合）、标记耦合（模块之间通过传递数据结构的指针或引用进行通信）、控制耦合（模块之间通过传递控制信号进行通信，一个模块控制另一个模块的执行）、公共耦合（多个模块共享同一个公共数据区域）、内容耦合（一个模块直接访问另一个模块的内部数据或代码，是最高级别的耦合）。在模块划分时，应尽量降低耦合度，避免公共耦合和内容耦合，优先采用数据耦合。模块划分的原则包括：

1. 单一职责原则：每个模块只负责一个特定的功能，避免一个模块承担多个不相关的功能。
2. 高内聚、低耦合原则：提高模块的内聚度，降低模块间的耦合度，确保模块的独立性。
3. 模块化原则：模块的大小要适中，过大的模块难以理解和维护，过小的模块会增加模块间的通信开销和管理难度。
4. 接口清晰原则：模块间的接口要清晰、明确，定义好接口的输入、输出和功能，避免接口模糊或歧义。
5. 可复用原则：模块的设计要考虑可复用性，便于在其他项目或系统中复用。

模块划分的方法主要有：功能分解法（按照软件的功能将系统分解为多个模块，每

个模块实现一个功能)、数据流分解法(根据数据流图,将系统分解为多个处理模块,每个模块对应一个处理过程)、面向对象分解法(按照系统中的对象将系统分解为多个模块,每个模块对应一个或多个对象)。

4.2.3 接口设计 接口是模块之间、系统与外部环境之间进行通信的通道,接口设计是概要设计的重要内容,其核心任务是定义模块之间的接口和系统与外部环境的接口,确保接口的清晰、明确、一致和可扩展。接口设计主要包括以下几个方面:

1. 模块间接口设计: 定义各个模块之间的接口,包括接口的名称、功能、输入参数、输出参数、数据类型、调用方式等。模块间接口应遵循高内聚、低耦合的原则,接口的设计要简洁、明确,避免模块间的直接依赖,确保模块的独立性。例如,一个用户管理模块和一个订单管理模块之间的接口,可定义为“根据用户 ID 查询用户信息”,输入参数为用户 ID,输出参数为用户的姓名、账号、联系方式等信息。
2. 系统与外部环境接口设计: 定义软件系统与外部环境(如其他系统、硬件设备、用户等)之间的接口,包括接口的类型、协议、数据格式、交互方式等。例如,软件系统与数据库之间的接口,需要定义数据库的连接方式、SQL 语句规范、数据交互格式等;软件系统与外部第三方系统之间的接

口，需要定义接口的调用协议（如 HTTP、TCP/IP）、数据格式（如 JSON、XML）、接口权限等。

3. 接口文档设计：编制接口文档，详细描述接口的相关信息，包括接口名称、功能描述、输入参数、输出参数、数据类型、调用方式、异常处理、注意事项等。接口文档是模块开发、测试和维护的重要依据，确保开发团队和相关人员对接口的理解一致。接口设计的原则包括：

1. 清晰性：接口的功能和参数要清晰、明确，避免模糊或歧义，便于开发人员理解和使用。
2. 一致性：接口的设计要遵循统一的规范和标准，包括命名规范、数据格式、调用方式等，确保接口的一致性。
3. 可扩展性：接口的设计要考虑未来的需求变化，预留扩展接口，便于后续的功能扩展，避免因需求变化导致接口的大规模修改。
4. 安全性：接口的设计要考虑安全性，对于敏感接口，需要进行权限控制、数据加密等处理，防止接口被非法调用或数据泄露。
5. 易用性：接口的设计要简单、易用，减少调用者的工作量，提高开发效率。

4.2.4 数据结构与数据库设计 数据结构与数据库设计是概要设计的重要组成部分，其核心任务是确定软件系统中需要存储的数据类型、数据结构和数据库结构，确保数据的存储安全、高效，便于数据的查询、插入、

修改和删除。

4.2.4.1 数据结构设计

数据结构是指数据的组织形式，包括数据的类型、存储方式、关联关系等。数据结构设计需要根据需求规格说明书中的数据需求，选择合适的数据结构，确保数据的存储和访问效率。常见的数据结构包括数组、链表、栈、队列、树、图、哈希表等，不同的数据结构具有不同的特点和适用场景：

1. 数组：适合存储一组相同类型的数据，访问速度快，但插入和删除操作效率低，适合数据量固定、访问频繁的场景。
2. 链表：适合存储动态变化的数据，插入和删除操作效率高，但访问速度慢，适合数据量不固定、插入删除频繁的场景。
3. 栈：遵循“先进后出”的原则，适合用于实现函数调用、表达式求值等场景。
4. 队列：遵循“先进先出”的原则，适合用于实现任务调度、消息队列等场景。
5. 树：适合用于存储具有层次关系的数据，如组织结构、文件系统等，查询效率高。
6. 图：适合用于存储具有复杂关联关系的数据，如社交网络、交通路线等。
7. 哈希表：适合用于快速查找和插入数据，通过哈希函数将数据映射到存储位置，访问效率高，适合数据量大、查询频繁的场景。

数据结构设计的原则包括：根据数据的使用场景选择合适的数据结构；确保数据的完整性和一致性；提高数据的访问效率和存储效率；

考虑数据的可扩展性。 4.2.4.2 数据库设计 数据库设计是指根据软件系统的数据需求,设计数据库的结构,包括数据库的选型、表结构设计、索引设计、约束设计等,确保数据库能够高效、安全地存储和管理数据。数据库选型需要根据软件的需求、规模、性能要求等因素,选择合适的数据库类型,常见的数据库类型包括关系型数据库和非关系型数据库:

1. 关系型数据库: 采用表格结构存储数据,数据之间具有明确的关联关系,支持 SQL 查询,适合存储结构化数据,如用户信息、订单信息等。常见的关系型数据库有 MySQL、Oracle、SQL Server 等。
2. 非关系型数据库: 不采用表格结构存储数据,数据格式灵活,支持多种数据类型,适合存储非结构化或半结构化数据,如图片、视频、日志等。常见的非关系型数据库有 MongoDB、Redis、Elasticsearch 等。

表结构设计是数据库设计的核心,需要根据数据需求,设计数据库表的结构,包括表名、字段名、字段类型、字段长度、主键、外键、约束等。表结构设计应遵循以下原则:

1. 范式原则: 遵循数据库设计范式(第一范式、第二范式、第三范式),避免数据冗余和数据不一致。第一范式要求每个字段都是原子的,不可再分;第二范式要求表中的每个非主键字段都完全依赖于主键;第三范式要求表中的每

个非主键字段都不传递依赖于主键。 2. 主键设计：每个表都应设置主键，主键用于唯一标识表中的每条记录，主键可以是单个字段，也可以是多个字段的组合（复合主键）。 3. 外键设计：通过外键建立表与表之间的关联关系，确保数据的完整性和一致性。外键是一个表中的字段，引用另一个表的主键。 4. 字段设计：字段的名称要清晰、规范，字段类型和长度要根据数据的实际情况选择，避免浪费存储空间；对于必填字段，要设置非空约束；对于具有唯一性要求的字段，要设置唯一约束。索引设计是提高数据库查询效率的重要手段，通过建立索引，可以加快数据的查询速度。索引设计应遵循以下原则：对查询频繁的字段建立索引；避免对频繁插入、修改、删除的字段建立过多索引；合理选择索引类型（如主键索引、唯一索引、普通索引等）。数据库设计的流程包括：需求分析（明确数据需求）、概念结构设计（绘制 ERD 图，描述数据之间的关系）、逻辑结构设计（将 ERD 图转化为数据库表结构）、物理结构设计（确定数据库的存储方式、索引设计等）、数据库实施（创建数据库、表、索引等）、数据库测试和维护。

4.3 详细设计 详细设计又称过程设计，是软件设计